

Pour les questions commençant par “*Présenter une solution*”, vous devez expliquer textuellement comment fonctionne votre solution. Pour les questions commençant par “*Donner un algorithme (ou une fonction)*”, vous devez donner l’algorithme (ou la fonction), avec des commentaires.

Pour la rédaction des algorithmes, vous ne supposerez pas disposer d’autres objets/fonctions que ceux donnés dans le document d’introduction au cours.

Attention, une réponse ne sera pas notée si:

- Le soin apporté à la rédaction et à la copie sont insuffisants,
- Toutes les réponses ne sont pas justifiées par un raisonnement clair et précis ou expliquées.

Exercice 1 : Graphe partiel sans suivant (9 points)

Soit un graphe orienté $G(V, E)$. Nous disons que l’arc $(w, x) \in E$ suit l’arc $(u, v) \in E$ si $v = w$, donc que le sommet final de l’arc (u, v) et le même que le sommet initial de l’arc (w, x) . Dans ce cas l’arc (w, x) est appelé le suivant de l’arc (u, v) . Par exemple, sur la figure 1, l’arc $(2, 4)$ est un suivant de l’arc $(0, 2)$.

Nous rappelons qu’un graphe partiel $GP = (VP, EP)$ d’un graphe $G = (V, E)$ est tel que $VP = V$ et $EP \subseteq E$, les sommets du graphe GP sont donc les mêmes que ceux du graphe G mais le graphe GP ne possède qu’une partie des arcs du graphe G .

Nous cherchons dans cet exercice à vérifier si un graphe possède un arc suivant et à trouver un graphe partiel maximal¹ ne contenant pas de suivant.

Question 1.1 : *Savoir si un graphe ne contient pas d’arc suivant*

1.a : *Présenter une solution pour déterminer si le graphe $GP = (V, EP)$ ne contient pas d’arc suivant*

¹Nous faisons ici la différence entre un ensemble sans suivant maximal, où aucun arc supplémentaire ne peut être ajouté à partir de l’ensemble initial des arcs sans ajouter un suivant, et un sous-ensemble maximum, le plus grand graphe partiel possible (le graphe partiel de cardinalité maximale) sans suivant.

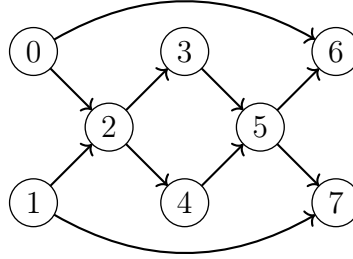


Figure 1: Graphe $G(V, E)$

1.b : Donner une fonction `sansSuivant`, qui prend en paramètre les tableaux FSP et APSP, et rend un booléen `vrai` si le graphe ne contient pas de suivant et `faux` sinon

Pour cette question, le graphe GP est donné dans les tableaux FSP, file des successeurs du graphe GP et APSP, adresse des premiers successeurs du graphe GP . Il faut donc que la fonction exploite au mieux ces structures de données.

1.c : Dérouter votre fonction sur le graphe partiel du graphe G donné dans les tableaux suivants:

FSP:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
2	6	-1	2	7	-1	-1	5	-1	5	-1	7	-1	-1	-1

APSP:

0	1	2	3	4	5	6	7
0	3	6	7	9	11	13	14

Question 1.2 : Algorithme de recherche d'un graphe partiel sans suivant

2.a : Donner un graphe partiel maximal sans arc suivant du graphe G (figure 1)

2.b : Présenter une solution pour la recherche d'un graphe partiel maximal GP sans arc suivant dans un graphe G , en justifiant la propriété maximale sur le graphe trouvé.

2.c : Donner une fonction `partielSansSuivant`, qui prend en paramètre les tableaux FS et APS, et rend les tableaux FSP et APSP d'un graphe partiel sans arc suivant

Pour cette question, le graphe G est donné dans les tableaux APS et FS. Il faut donc que la fonction exploite au mieux ces structures de données.

2.d : D  rouler votre algorithme sur le graphe de la figure 1 et donner le r  sultat

Correction Exercice 1 : Graphe partiel sans suivant

Solution question 1.1 : Savoir si un graphe ne contient pas d'arc suivant

Solution question 1.a : Pr  senter une solution pour d  terminer si le graphe $GP = (V, EP)$ ne contient pas d'arc suivant

Pour qu'un graphe ne contienne pas d'arc suivant il faut qu'aucune des extr  mit  s finales d'un arc ne soit l'extr  mit   initiale d'un autre. Cela signifie que, si sommet v est le successeur d'un sommet u , par l'arc (u, v) du graphe alors v ne peut pas avoir de successeur.

Solution question 1.b : Donner une fonction `sansSuivant`, qui prend en param  tre les tableaux FSP et $APSP$, et rend un bool  en `vrai` si le graphe ne contient pas de suivant et `faux` sinon

Le graphe    tester est donn   dans une structure de type FS/APS. Pour savoir si le graphe poss  de au moins un arc suivant, il est possible de prendre, pour chaque sommet de G , ses successeurs et de v  rifier que ceux-ci n'ont eux-m  mes pas de successeur.

Algorithme 1 : Recherche d'arc suivant dans un graphe donn   par FSP et $APSP$

```

1 Fonction sansSuivant(FSP,APSP):
2 Donn  es :
3  $M$ : taille du tableau  $FSP$ 
4 suivant: bool  en, init faux
5 d  but
6    $M \leftarrow FSP.lentgh$ 
7   pour  $i \leftarrow 0$      $M - 1$  faire
8     si  $FSP[i] \neq -1$  alors
9        $j \leftarrow APSP[FSP[i]]$ 
10      si  $FSP[j] \neq -1$  alors retourner faux
11 retourner vrai
```

L'algorithme 1 donne le code de la fonction `sansSuivant(FSP, APSP)`. Il parcourt tous les arcs du graphe    travers le `pour` (ligne 7) sur le tableau FSP . Pour chacun des successeurs, il trouve le point de d  part ($APSP[FSP[j]]$),

ligne 9) dans le tableaux FSP des successeurs du sommet. Si ce point de départ n'est pas un marqueur (ligne 10) cela signifie que ce successeur a lui-même un successeur et la fonction a trouvé un suivant. Elle retourne donc **faux**. Si tous les sommets ont été parcouru sans trouver de suivant (sans sortir à la ligne 10), cela signifie qu'il n'y a pas d'arc suivant de le graphe et la fonction retourne **vrai**.

Solution question 1.c : *Dérouler votre fonction sur le graphe partiel du graphe G donné dans les tableaux suivants:*

La fonction parcourt le tableau FSP .

Pour $FSP[0]$, 2 est le premier successeur du sommet 0, $APSP[2] = 6$, et $FSP[6] = -1$, l'algorithme poursuit la boucle pour.

Pour $FSP[1]$, 6 est successeurs de 0, $APSP[6] = 13$, et $FSP[13] = -1$, l'algorithme poursuit la boucle.

Pour $FSP[2]$, c'est un marqueur, l'algorithme poursuit la boucle.

Pour $FSP[3]$, 2 est successeurs de 1, $APSP[2] = 6$, et $FSP[2] = -1$, l'algorithme poursuit la boucle.

Pour $FSP[4]$, 7 est le successeurs de 1, $APSP[7] = 14$, et $FSP[14] = -1$, l'algorithme poursuit la boucle.

Pour $FSP[5]$ et $FSP[6]$, ce sont des marqueurs, l'algorithme poursuit la boucle.

Pour $FSP[7]$, 5 est le successeurs de 3, $APSP[5] = 11$, et $FSP[11] = -7$, ce qui donne 1 succeseurs pour le sommet 5, lui même successeur de 3, la fonction a trouvé un arc suivant et retourne *faux*.

Solution question 1.2 : *Algorithme de recherche d'un graphe partiel sans suivant*

Solution question 2.a : *Donner un graphe partiel maximal sans arc suivant du graphe G (figure 1)*

L'ensemble d'arcs $\{(0, 6), (0, 2), (1, 7), (1, 2), (3, 5), (4, 5)\}$ constitue un ensemble maximal car quelque soit l'arc du graphe G qui ne fait pas partie de cet ensemble, si nous l'ajoutons à l'ensemble alors nous créons un voisin dans le graphe.

Solution question 2.b : *Présenter une solution pour la recherche d'un graphe partiel maximal sans arc suivant GP dans un graphe G , en justifiant la propriété maximale sur le graphe trouvé.*

Une première solution simple pour répondre à cette question est de partir

d'un graphe sans arc ($EP = \emptyset$), puis d'itérer sur les arcs du graphe. Pour chaque arc, un nouveau graphe FSP et $APSP$ est créé et l'algorithme utilise la fonction `sansSuivant`. Si l'arc ne crée pas de suivant l'algorithme continue avec des tableaux FSP et $APSP$ contenant cet arc. Si cet arc crée un suivant, il n'est pas ajouté et l'algorithme reprend avec les tableaux FSP et $APSP$ précédents. Cependant, cet algorithme peut être lourd si nous sommes obligés de recréer de nouvelles structures de données à chaque itération. Dans ce cas il est possible de trouver une solution pour faire évoluer la structure de données et de refaire sur cette structure de données ce qui est fait dans l'algorithme `sansSuivant`.

Une solution simple, qui exploite les tableaux APS et FS , consiste à prendre les arcs les uns après les autres et à ne les ajouter aux tableaux finals FSP et $APSP$ que s'ils ne créent pas de suivant dans ce graphe. Pour cela nous vérifions si le sommet initial de l'arc a un prédécesseur, donc si un sommet a déjà un arc qui arrive au sommet courant, et si le sommet final de l'arc a un successeur, donc si ce sommet final a un arc qui en part. Pour chaque ajout d'arc nous conservons l'information du successeur, le sommet initial, et du prédécesseur, le sommet final, qu'il crée.

Pour justifier la propriété maximale du graphe trouvé, nous pouvons montrer que chaque fois qu'un arc est ajouté il ne crée pas d'arc suivant car les arcs qui ne sont pas ajoutés ne l'ont pas été car ils auraient créé des suivants. Comme nous parcourons tous les arcs du graphe initial, nous arrivons donc bien à la propriété de maximalité, tout autre arc ajouté fait que le graphe n'est plus sans suivant.

Attention à l'approche inverse qui consiste à enlever des arcs. Si vous l'appliquez sur le graphe de la figure 1, vous verrez que cela ne permet pas d'obtenir un graphe maximal car, en enlevant certains arcs, d'autres qui ont été enlevés avant auraient pu être laissés.

Solution question 2.c : Donner une fonction `partielSansSuivant`, prend en paramètre les tableaux FS et APS , et rend les tableaux FSP et $APSP$ d'un graphe partiel sans arc suivant

Algorithme 2 : Recherche d'un graphe partiel sans suivant dans un graphe donné par FS et APS

```

1  Fonction partielSansSuivant( $FS, APS$ ):
2  Données :
3   $N$ : entier, nombre de sommet, init  $APS.length$ 
4   $M$ : entier, nombre d'arcs, init  $FS.length$ 
5   $FSP$ : tableau d'entiers, de longueur  $M$ 
6   $APSP$ : tableau d'entiers, de longueur  $N$ 
7   $pred$ : tableau de booléens, de longueur  $N$ 
8   $succ$ : tableau de booléens, de longueur  $N$ 
9   $u$ : entier, sommet, init 0
10  $ip$ : entier, position dans  $FSP$ , init 0
11 début
12   pour  $i \leftarrow 0$  à  $N$  faire
13      $pred[i] \leftarrow faux$ 
14      $succ[i] \leftarrow faux$ 
15    $APSP[u] \leftarrow 0$ 
16   pour  $i \leftarrow 0$  à  $M - 1$  faire
17     si  $FS[i] \neq -1$  alors
18       si  $pred[u] = faux$  alors
19          $v \leftarrow FS[i]$ 
20         si  $succ[u] = faux$  alors
21           // ajout de l'arc
22            $FSP[ip] \leftarrow v$ 
23            $ip \leftarrow ip + 1$ 
24            $succ[u] \leftarrow vrai$ 
25            $pred[v] \leftarrow vrai$ 
26       sinon
27         // change de sommet
28          $FSP[ip] \leftarrow -1$ 
29          $ip \leftarrow ip + 1$ 
30         si  $u < N - 1$  alors
31            $u \leftarrow u + 1$ 
32            $APSP[u] \leftarrow ip$ 
33   retourner ( $FSP, APSP$ )

```

L'algorithme 2 donne le code d'une fonction `partielSansSuivant(FS, APS)`. Il reprend largement la structure de l'algorithme 1, à la différence

près qu'il génère, en même temps qu'il parcourt les tableaux FS et APS , de nouveaux tableaux FSP et $APSP$.

L'algorithme débute en initialisant les tableaux $pred$ et $succ$ (lignes 13 et 14) à faux. Pour mémoriser, respectivement, les sommets qui ont au moins un prédécesseur et au moins un successeur.

L'algorithme boucle ensuite sur l'ensemble des arcs du graphe G (ligne 15 et 17). Pour pouvoir ajouter un arc, il faut que ni son sommet initial u n'ait de prédécesseur, ni son sommet final v n'ait de successeur. Si c'est le cas, l'arc est ajouté au tableau FSP final (lignes 20 et 21) et les sommets sont ajoutés dans les tableaux $pred$ et $succ$ pour qu'ils ne puissent plus être utilisés pour ajouter un successeur, pour v , ou un prédécesseur, pour u . Sinon l'arc n'est pas ajouté.

Lorsque une valeur -1 est rencontrée dans le tableau, il ne s'agit pas d'un arc mais d'un marqueur de changement de sommet. Le marquer est reporté dans le tableau FSP et le numéroté de sommet incrémenté (ligne 25 à 27) puis le tableau $APSP$ est mis à jour pour le sommet suivant, si le sommet en cours n'est pas le dernier sommet (lignes 27 à 29).

À noter que la taille des tableaux FSP et $APSP$ est la même que celle des tableaux FS et APS . C'est de toutes façons le cas du tableau $APSP$ puisqu'il n'y a pas de suppression de sommet, mais la taille du tableau FSP est forcément inférieure à celle du tableau FS puisqu'on supprime des arcs dans le graphe initial. Cette taille permet cependant d'être sûr d'avoir assez de place pour tous les arcs si le graphe initial est sans suivant. Il serait éventuellement possible, une fois le tableau FSP généré d'en créer un nouveau d'une taille adaptée (jp) et d'y recopier le tableau FSP .

Solution question 2.d : Dérouler votre algorithme sur le graphe de la figure 1 et donner le résultat

Pour faciliter le déroulement de l'algorithme nous donnons les structures FS et APS du graphe de la figure 1.

FS:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
2	6	-1	2	7	-1	3	4	-1	5	-1	5	-1	6	7	-1	-1	-1

APS:

0	1	2	3	4	5	6	7
0	3	6	9	11	13	16	17

Initialement, les tableaux FSP et $APSP$ n'ont pas de valeur, sauf $APSP[0]$ qui est mis à 0.

FSP:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17

APS:	0	1	2	3	4	5	6	7
	0							

Les tableaux *pred* et *succ* sont initialisés à faux.

pred:

0	1	2	3	4	5	6	7
F	F	F	F	F	F	F	F

succ:

0	1	2	3	4	5	6	7
F	F	F	F	F	F	F	F

Pour le premier arc ($i = 0$), le sommet initial u est 0 et il possède des arcs sortants sans avoir de prédécesseur. Le premier vers 2 qui n'a pas de successeur, l'arc peut donc être ajouté. De même pour le second arc qui a 6 pour sommet final. La troisième entrée de *FS* est à -1 , on passe par la partie sinon de l'algorithme donc du sommet 0 au sommet 1 et l'état des tables est le suivant:

FSP:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	2	6	-1															

APS:	0	1	2	3	4	5	6	7
	0	3						

pred:

0	1	2	3	4	5	6	7
F	F	T	F	F	F	T	F

succ:

0	1	2	3	4	5	6	7
T	F	F	F	F	F	F	F

Pour le sommet 1 le traitement est le même, ajout de deux arcs et du marqueur, et l'état des tables est le suivant:

FSP:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	2	6	-1	2	7	-1												

APS:	0	1	2	3	4	5	6	7
	0	3	6					

pred:

0	1	2	3	4	5	6	7
F	F	T	F	F	F	T	T

succ:

0	1	2	3	4	5	6	7
T	T	F	F	F	F	F	F

Le sommet 2 ($i = 6$) a au moins un prédécesseur, ses arcs sont donc parcourus sans être ajoutés et l'état des tables est le suivant:

FSP:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
2	6	-1	2	7	-1	-1											

APS:

0	1	2	3	4	5	6	7
0	3	6	7				

pred:

0	1	2	3	4	5	6	7
F	F	T	F	F	F	T	T

succ:

0	1	2	3	4	5	6	7
T	T	F	F	F	F	F	F

Les sommets 3 et 4 n'ont pas de prédécesseur et conduisent au sommet 5 qui n'a pas de successeur. Après le parcours de leurs arcs, l'état des tables est le suivant:

FSP:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
2	6	-1	2	7	-1	-1	5	-1	5	-1							

APS:

0	1	2	3	4	5	6	7
0	3	6	7	9	11		

pred:

0	1	2	3	4	5	6	7
F	F	T	F	F	T	T	T

succ:

0	1	2	3	4	5	6	7
T	T	F	T	T	F	F	F

Le sommet 5 a au moins un prédécesseur, ses arcs ne sont pas ajoutés dans les tables. Le sommet 6 ($i = 16$) ne possède pas d'arc. Pour le sommet 7 ($i = 17 = M - 1$) nous avons $u = N - 1$ donc le marqueur est reporté dans le tableau *FSP* mais le tableau *APSP* n'est pas modifié. L'algorithme termine et l'état final des tables *FSP* et *APSP* est le suivant:

FSP:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
2	6	-1	2	7	-1	-1	5	-1	5	-1	-1	-1					

APS:

0	1	2	3	4	5	6	7
0	3	6	7	9	11	16	17

Nous obtenons alors le graphe 2 et nous pouvons constater que celui-ci est bien maximal car l'ajout d'un des arcs enlevés $((2, 3), (2, 4), (5, 6), (5, 7))$ crée un suivant.

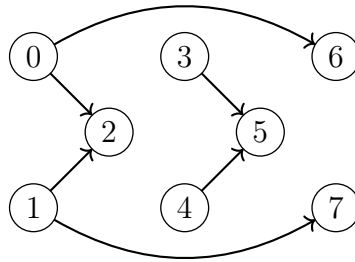


Figure 2: Graphe $G(V, E)$

Exercice 2 : Recherche des isthmes d'un graphe (6 points)

Nous rappelons que, dans un graphe non orienté, un isthme est une arête qui fait augmenter le nombre de composantes connexes quand elle est supprimée. Enlever cette arête “coupe” le graphe en deux. La figure 3 présente deux graphes possédant un isthme: entre les sommets 3 et 4 pour le premier et entre les sommets 5 et 6 pour le second.

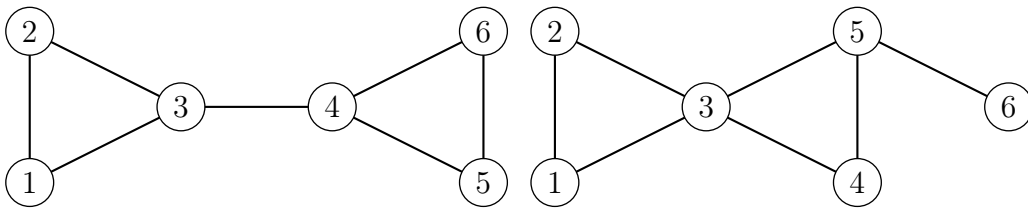


Figure 3: Exemples de graphe possédant un isthme

Pour la suite, nous considérons un graphe $G = (V, E)$ non orienté connexe, donné dans une matrice d'adjacence, codée dans le tableau à deux dimensions M .

Question 2.1 : *Savoir si l'arête $\{x, y\}$ du graphe G est un isthme*

1.a : *Présenter une solution générique permettant de savoir si l'arête $\{x, y\}$ du graphe G , sans tenir compte de la structure de donnée de codage.*

1.b : *Donner une fonction `estUnIsthme(x,y,M)` rendant vrai si l'arête $\{x, y\}$ passée en paramètre est un isthme, faux sinon*

Question 2.2 : *Obtenir la liste des isthmes du graphe G*

2.a : *Présenter une solution générique permettant d'obtenir la liste des isthmes du graphe G , sans tenir compte de la structure de donnée de codage*

2.b : *Donner une fonction `listeIsthmes(M)` retournant la liste L des isthmes du graphe G .*

Correction Exercice 2 : Recherche des isthmes d'un graphe

Solution question 2.1 : *Savoir si l'arête a du graphe G est un isthme*

Solution question 1.a : *Présenter une solution générique permettant de savoir si l'arête a du graphe G , sans tenir compte de la structure de donnée de codage.*

On considère le graphe G' sans cette arête et on fait un parcours du graphe depuis n'importe quel sommet, si le parcours n'atteint pas tous les sommets du graphe, alors l'arête était un isthme.

Solution question 1.b : *Donner une fonction `estUnIsthme(x,y,M)` rendant vrai si l'arête passée en paramètre est un isthme, faux sinon*

Pour implémenter la méthode précédente avec une matrice d'adjacence, il faut disposer d'une fonction de parcours en profondeur du graphe qui compte le nombre de sommets visités. Nous pouvons ensuite parcourir la matrice M en enlevant les arêtes une par une et en appelant la fonction du parcours en profondeur en vérifiant le nombre de sommets visités.

Algorithme 3 : Fonction de parcours en profondeur d'un graphe rendant le nombre de sommets parcourus

```
1 Fonction parcoursProfMA( $x, y, M$ )
2 Données :
3 nbSommetsParcours: entier
4  $p$ : la pile des sommets dans l'ordre de leur rencontre, initialisé à vide
5 couleur[ $u$ ]:  $u \in \{BLANC, GRIS\}$ , init à BLANC,  $\forall u \in V$ 
6 début
7    $p.push(0)$ 
8    $nbSommetsParcours \leftarrow 1$ 
9   tant que  $\neg p.empty()$  faire
10      $u \leftarrow p.pop()$ 
11     pour  $v \leftarrow 0$  à  $n - 1$  faire
12       si  $(M[u, v] = 1) \wedge (couleur[v] = BLANC)$  alors
13          $couleur[v] \leftarrow GRIS$ 
14          $p.push(v)$ 
15          $nbSommetsParcours \leftarrow nbSommetsParcours + 1$ 
16   retourner  $nbSommetsParcours$ 
```

La fonction `parcoursProfMA` donnée par l'algorithme 3 réalise un parcours

en profondeur sur un graphe donné dans une matrice d'adjacence et retourne le nombre de sommets parcourus. Cette fonction est adaptée à partir de l'algorithme du cours. Le principal changement est le parcours des voisins d'un sommet (lignes 11 et 12), où le parcours des voisins est réalisé à partir des valeurs à 1 de la ligne de la matrice correspondante au sommet u en cours.

Algorithme 4 : Fonction permettant de savoir si une arête $\{x, y\}$ est un isthme dans le graphe donné par M

```

1 Fonction estIsthme( $x, y, M$ )
2 Données :
3 nbSommetsParcourus: entier
4  $n$ : entier init nombre de sommets du graohe
5 début
6   si  $M[x, y] \neq 1$  alors retourner faux
7    $M[x, y] = 0$ 
8    $M[y, x] = 0$ 
9    $nbSommetsParcourus = \text{parcoursProfMA}(M)$ 
10   $M[x, y] = 1$ 
11   $M[y, x] = 1$ 
12  si  $nbSommetsParcourus = n$  alors retourner faux
13  retourner vrai
```

La fonction **estIsthme** donnée par l'algorithme 4 permet de savoir si l'arête $\{x, y\}$ est un isthme dans le graphe donné par M . Pour cela elle vérifie déjà $\{x, y\}$ est bien une arête du graphe (ligne 6). Elle supprime ensuite l'arête de la matrice (ligne 7 et 8) et fait appel, avec ce nouveau graphe, à la fonction **parcoursProfMA** définie précédemment. Si le nombre de sommets visité par la fonction est égal à celui du graphe initial (ligne 9), alors cela signifie que l'arête n'est pas un isthme et la fonction retourne **faux**. Dans le cas contraire, elle retourne **vrai**. Suivant la nature du passage de la matrice à la fonction, il peut être nécessaire de remettre l'arête dans la matrice (lignes 11 et 12). Pour retirer ou remettre une arête dans un matrice d'adajacence, il est nécessaire de le faire de manière symétrique ($\{x, y\}$ et $\{y, x\}$) car l'arête apparaît deux fois dans la matrice.

Solution question 2.2 : *Obtenir la liste des isthmes du graphe G*

Solution question 2.a : *Présenter une solution générique permettant d'obtenir la liste des isthmes du graphe G , sans tenir compte de la structure de donnée de codage*

Pour trouver toutes les isthmes d'un graphe, il suffit d'appliquer la méthode précédente à chacune des arêtes du graphes.

Solution question 2.b : Donner une fonction `listeIsthmes(M)` retournant la liste L des isthmes du graphe G .

L'algorithme 5 applique la fonction `estIsthme` à chacune des arêtes du graphe. À noter que nous avons utilisé une numérotation des sommets de 0 à $n - 1$ pour plus de simplicité.

Algorithme 5 : Algorithme de recherche des isthmes dans un graphe

```

1 Fonction listeIsthmes(M) Données :
2  $L$ : la liste des isthmes, init listeVide()
3  $isthme$ : booléen
4 début
5   pour  $i \leftarrow 0$  à  $n - 1$  faire
6     pour  $j \leftarrow i + 1$  à  $n - 1$  faire
7       si  $M[i, j] = 1$  alors
8          $isthme = estIsthme(i, j, M)$ 
9         si  $isthme$  alors  $L.add(i, j)$ 
10  retourner  $L$ 

```

La fonction `listeIsthmes(M)` présentée dans l'algorithme 5 permet d'obtenir la liste de arêtes qui sont des isthmes dans le graphe donné par la matrice M . La fonction parcourt la matrice (lignes 4 et 5) et teste chacune des arêtes avec la fonction `estIsthme`. Si la fonction retourne `vrai`, elle ajoute l'arête à la liste. À noter que le parcours au dessus de la diagonale de la matrice, en excluant cette dernière, est suffisant puisque la matrice d'un graphe non orienté est

Exercice 3 : Confection de gâteaux (5 points)

Pour son anniversaire Hilal veut faire des gâteaux. Il dispose de deux recettes, la première permet de faire un petit gâteau de 6 parts et la seconde permet de faire un plus gros gâteau de 10 parts. La première recette nécessite 1 œuf, 100 gr. de sucre et 200 gr. de farine. La seconde recette nécessite 1 œuf, 200 gr. de sucre et 300 gr. de farine.

En regardant dans ses placards Hilal trouve 7 œufs, 1 kg. de sucre et 1.8 kg. de farine. Pour satisfaire ses invités Hilal souhaite faire le plus grand nombre de parts de gâteau avec ce qu'il a trouvé.

Le tableau 1 synthétise les données du problème.

	œuf	sucre	farine	parts
recette petit gâteau	1	100 gr.	200 gr.	6
recette grand gâteau	1	200 gr.	300 gr.	10
	7	1 kg	1.8 kg	

Table 1: Données du problème de fabrication des gâteaux

Question 3.1 : *Écrire le programme linéaire permettant de trouver le plus grand nombre de parts de gâteau.*

Définir l'ensemble des variables permettant de décrire le problème. Donner l'expression de l'objectif à maximiser. Donner les contraintes encadrant ce problème d'optimisation. Les étapes de rédaction des différentes (in)équations doivent être clairement expliquées.

Question 3.2 : *Simplifier le problème.*

Expliquer comment vous pouvez simplifier certaines inéquations et sous quelle condition.

Question 3.3 : *Résoudre le problème avec la méthode graphique.*

Question 3.4 : *Résoudre le problème avec la méthode du simplexe.*

Correction Exercice 3 : Programmation linéaire

Solution question 3.1 : *Écrire le programme linéaire permettant de trouver le plus grand nombre de parts de gâteau.*

On pose x_1 le nombre de petit gâteaux et x_2 le nombre de gros gâteaux. Puisque nous souhaitons maximiser le nombre de parts, qu'un petit gâteau donne 6 parts et un gros 10 parts, la fonction à maximiser est: $6x_1 + 10x_2$.

En traduisant les quantités disponibles sous la forme de contraintes, nous obtenons le programme linéaire suivant pour les valeurs données dans l'énoncé:

$$\left\{ \begin{array}{ll} \text{maximiser} & 6x_1 + 10x_2 \\ \text{avec les contraintes} & \begin{array}{l} \text{œuf : } x_1 + x_2 \leq 7 \\ \text{sucre : } 100x_1 + 200x_2 \leq 1000 \\ \text{farine : } 200x_1 + 300x_2 \leq 1800 \end{array} \\ \text{et} & x_1, x_2 \geq 0 \end{array} \right.$$

Solution question 3.2 : *Simplifier le problème.*

Il est possible de simplifier les coefficients des équations *sucre* et *farine* en les divisant par 100. Nous obtenons:

$$\left\{ \begin{array}{ll} \text{maximiser} & 6x_1 + 10x_2 \\ \text{avec les contraintes} & \begin{array}{l} \text{œuf : } x_1 + x_2 \leq 7 \\ \text{sucre : } x_1 + 2x_2 \leq 10 \\ \text{farine : } 2x_1 + 3x_2 \leq 18 \end{array} \\ \text{et} & x_1, x_2 \geq 0 \end{array} \right.$$

Solution question 3.3 : *Résoudre le problème avec la méthode graphique.*

Nous posons $x_1 = x$ et $x_2 = y$ et nous obtenons les équations de droites suivantes:

- $D_1 : y = 7 - x$
- $D_2 : y = 5 - 1/2x$
- $D_3 : y = 6 - 2/3x$

Pour finir, la droite de la fonction à optimiser a pour équation: $y = b - 3/5x$.

Les équations de ces droites étant simples, il est plus facile de réaliser une résolution graphique. La figure 4 donne la représentation des droites. Nous pouvons constater que la droite $y = b - 3/5x$ maximise le nombre de parts à l'intersection des droites D_1 et D_2 pour 4 petits gâteaux et 3 gros. Hilal obtient ainsi 54 parts de gâteaux.

Solution question 3.4 : *Résoudre le problème avec la méthode du simplexe.*

Le problème est formalisé sous forme canonique:

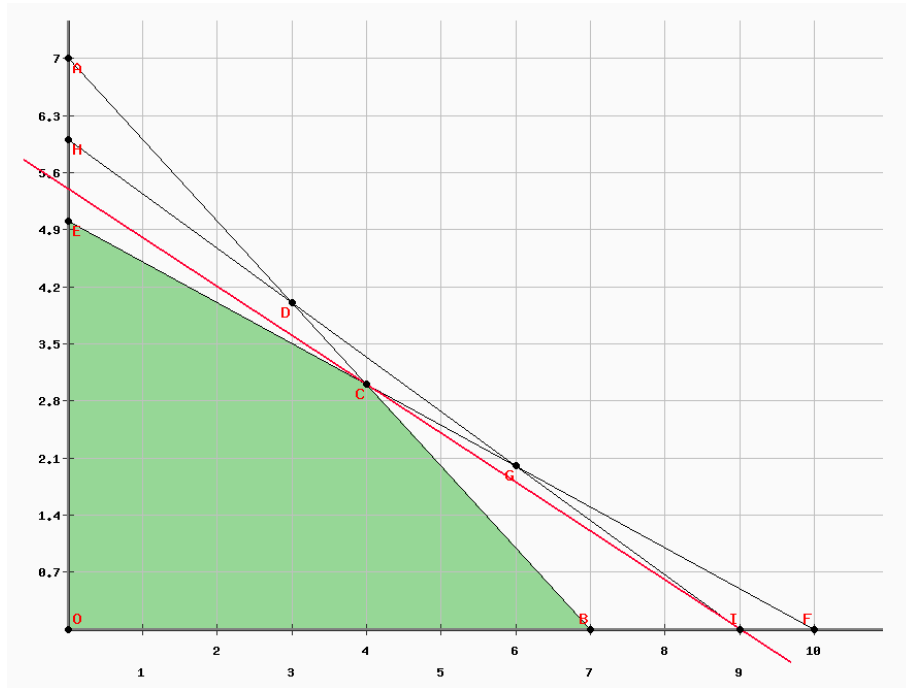


Figure 4: Représentation graphique du programme linéaire donné dans le tableau

$$\left\{ \begin{array}{ll} \text{maximiser} & 6x_1 + 10x_2 \\ \text{avec les contraintes} & \begin{array}{l} x_1 + x_2 \leq 7 \\ x_1 + 2x_2 \leq 10 \\ 2x_1 + 3x_2 \leq 18 \end{array} \\ \text{et } x_1, x_2 \geq 0 \end{array} \right.$$

Ici il pourrait être utile de voir que la somme des deux première contraintes conduit à contrainte suivante:

$$2x_1 + 3x_2 \leq 17$$

De ce fait la troisième contrainte n'est pas utile puisque les deux premières sont plus contraignantes. Celà est d'ailleurs visible sur la figure 4 où nous voyons que la droite passant par les points I et H ne participe pas à définir l'espace des solutions possibles.

Il est passé sous forme standard en introduisant les variables d'écart x_3 , x_4 et x_5 .

$$\left\{ \begin{array}{l} \text{maximiser} \quad z = 2x_1 + 3x_2 \\ \text{avec les contraintes} \quad x_3 = 7 - x_1 - x_2 \\ \quad \quad \quad x_4 = 10 - x_1 - 2x_2 \\ \quad \quad \quad x_5 = 18 - 2x_1 - 3x_2 \end{array} \right.$$

Nous pouvons maintenant résoudre ce programme par l'algorithme du simplexe.

Avec notre programme les solutions de base sont:

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}, \overline{x_5}) = (0, 0, 7, 10, 18)$$

et la valeur de l'objectif $z = 0$.

La variable x_2 est la variable hors-base qui a le plus grand coefficient dans la fonction objectif, c'est elle dont la valeur est augmentée en premier. Dans les contraintes nous avons:

$$\begin{aligned} x_2 > 7 &\Rightarrow x_3 < 0 \\ x_2 > 10/2 = 5 &\Rightarrow x_4 < 0 \rightarrow \text{contrainte la plus stricte} \\ x_2 > 18/3 = 9 &\Rightarrow x_5 < 0 \end{aligned}$$

C'est la deuxième contrainte qui est la plus stricte. Nous exprimons donc x_2 dans cette équation:

$$x_2 = 5 - \frac{1}{2}x_1 - \frac{1}{2}x_4$$

Ceci qui nous donne le programme suivant en remplaçant x_2 :

$$\left\{ \begin{array}{l} z = 6x_1 + 10(5 - \frac{1}{2}x_1 - \frac{1}{2}x_4) \\ x_3 = 7 - x_1 - (5 - \frac{1}{2}x_1 - \frac{1}{2}x_4) \\ x_2 = 5 - \frac{1}{2}x_1 - \frac{1}{2}x_4 \\ x_5 = 18 - 2x_1 - 3(5 - \frac{1}{2}x_1 - \frac{1}{2}x_4) \end{array} \right.$$

Et se simplifie en:

$$\left\{ \begin{array}{l} z = 50 + x_1 - 5x_4 \\ x_3 = 2 - \frac{1}{2}x_1 + \frac{1}{2}x_4 \\ x_2 = 5 - \frac{1}{2}x_1 - \frac{1}{2}x_4 \\ x_5 = 3 - \frac{1}{2}x_1 + \frac{3}{2}x_4 \end{array} \right.$$

Pour ce nouveau programme, la solution avec les variables hors-base nulles est:

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}, \overline{x_5}) = (0, 5, 2, 0, 3)$$

Et la valeur objectif $z = 50$.

Cette fois c'est au tour de x_1 d'entrer en base. Les contraintes sont maintenant:

$$\begin{aligned} x_1 > 4 &\Rightarrow x_3 < 0 \rightarrow \text{contrainte la plus stricte} \\ x_1 > 10 &\Rightarrow x_2 < 0 \\ x_1 > 6 &\Rightarrow x_5 < 0 \end{aligned}$$

C'est la première équation qui est la plus contraignante et qui donne l'expression de x_1 .

$$x_1 = 4 - 2x_3 + x_4$$

Ceci qui nous donne le programme suivant en remplaçant x_2 :

$$\begin{cases} z = 50 + (4 - 2x_3 + x_4) - 5x_4 \\ x_1 = 4 - 2x_3 + x_4 \\ x_2 = 5 - \frac{1}{2}(4 - 2x_3 + x_4) - \frac{1}{2}x_4 \\ x_5 = 3 - \frac{1}{2}(4 - 2x_3 + x_4) + \frac{3}{2}x_4 \end{cases}$$

Et se simplifie en:

$$\begin{cases} z &= 54 &- 2x_3 &- 4x_4 \\ x_1 &= 4 &- 2x_3 &+ x_4 \\ x_2 &= 3 &+ x_3 &- x_4 \\ x_5 &= 1 &+ x_3 &+ x_4 \end{cases}$$

Il n'y a plus de variable à coefficient positif dans l'équation, nous sommes donc arrivés au bout de la maximisation avec:

$$(\overline{x_1}, \overline{x_2}, \overline{x_3}, \overline{x_4}) = (4, 3, 0, 0, 1)$$

Et la valeur objectif $z = 54$ et nous retrouvons bien les résultats obtenus avec la méthode graphique.